

From Unstructured Recall to Schema-Grounded Memory: Reliable AI Memory via Iterative, Schema-Aware Extraction

Alex Petrov¹, Alexander Gusak¹, Denis Mukha¹, and Dima Korolev¹

¹xmemory, [alex, alexg, denis, dima]@xmemory.ai

August 14, 2026

Abstract

Persistent AI memory is often reduced to a retrieval problem: store prior interactions as text, embed them, and ask the model to recover relevant context later. The approach is brittle because every retrieval re-runs inference on raw text, making it hard to enforce consistency, track state, or answer precise questions reliably over time. Small extraction errors compound silently into incorrect behavior. The memory workloads that matter most in production—exact fact lookup, state tracking, aggregation, relational queries, and explicit unknowns—require memory to behave less like search and more like a system of record.

Reliable external AI memory must therefore be schema-grounded. Schemas define what must be remembered, what may be ignored, and which values must never be inferred—turning relevance from an implicit, read-time heuristic into an explicit, enforceable contract. **xmemory** realises this design by converting unstructured interaction logs into validated, schema-conformant records via an iterative, schema-aware write path. The pipeline decomposes ingestion into object detection, field detection, and field-value extraction, with validation gates, local retries, an LLM judge-in-the-loop, and stateful prompt control. The result shifts interpretation to the write path: reads become constrained queries over verified records rather than repeated inference over retrieved prose.

Evaluation on structured extraction and end-to-end memory benchmarks confirms the advantage. On the extraction benchmark, the judge-in-the-loop configuration reaches 90.42% object-level accuracy and 62.67% output accuracy, above all tested frontier structured-output baselines. On the end-to-end memory benchmark, **xmemory** reaches 97.10% F1, compared with 80.16%–87.24% across the third-party baselines. On the application-level task, **xmemory** reaches 95.2% accuracy, outperforming specialised memory systems, code-generated Markdown harnesses, and customer-facing frontier-model application harnesses. The results show that, for memory workloads requiring stable facts and stateful computation, architecture matters more than retrieval scale or model strength alone.

Contents

1	The memory problem in AI systems	3
1.1	Memory query types exceed what semantic retrieval can support	3
1.2	Summarisation and embedding-based recall introduce unavoidable loss	4
2	Why unstructured memory fails at factual recall	4
2.1	Implicit relevance vs explicit relevance	4
2.2	Limits of semantic similarity for precise questions	5
2.3	Why scale, reranking, or hybrid retrieval do not fix factual recall	5

3	Schema-grounded memory as explicit relevance definition	6
3.1	Schemas as contracts for what must be remembered	6
3.2	Memory as facts, not text	7
4	Architectural tradeoff: write-path complexity vs read-path simplicity	7
4.1	Unstructured retrieval (RAG-style) vs structured memory	7
4.2	Graph RAG: introducing partial structure into retrieval	8
4.3	Why not a general-purpose relational database?	9
4.4	Text-to-SQL as constrained retrieval	9
5	Empirical limits of single-pass structured output	10
5.1	Field accuracy vs object accuracy	10
5.2	Compounding loss and memory corruption	11
6	Iterative, schema-aware extraction	12
6.1	Overall architecture: object detection, field detection, field value extraction . . .	12
6.2	Prompt engine role	13
6.3	Memory contexts: request, session, and main	14
6.4	Real-workflow decision latency and token consumption	16
7	Schema lifecycle: bootstrapping and evolution	17
7.1	Bootstrapping the schema	17
7.2	Agent-assisted schema design from intended questions	17
7.3	Schema evolution from observed usage	17
8	Evaluation: memory quality improves with structure	18
8.1	Benchmarks	18
8.2	Extraction results and comparison	19
8.3	Tests of memory functions and comparison with other memory systems	21
8.4	Model real-life use case experiment	25
9	Limitations	26
10	Conclusions and future work	26
A	Appendix	31
A.1	Information-theoretic intuition	31
A.2	Overview of existing memory benchmarks: LoCoMo and LongMemEval	31

1 The memory problem in AI systems

1.1 Memory query types exceed what semantic retrieval can support

The term “memory” covers several mechanisms in AI systems, and this paper addresses only one of them [1, 2]. *Parametric memory* is knowledge encoded in model weights through training or fine-tuning: it is implicit, hard to inspect, and cannot reliably be updated record by record at runtime in current production systems. *In-context memory* is information present in the active context window, including conversation turns, retrieved passages, or scratchpad state; it is transient, bounded by context length, and lost when the session ends. *Latent memory* refers to compressed state in hidden activations or key-value caches that can shape generation without being explicitly addressable.

This paper focuses on *external, explicit memory*: a persistent store outside the model, written at runtime and queried later to supply facts to downstream reasoning. External memory is the right architectural locus for addressable facts, stable state across sessions, and deterministic computation over stored records. The question is therefore narrow: given that persistent factual memory must live outside the model, what internal design does it need to be reliable?

Scope. We do not claim that schema-grounded memory is optimal for every memory use case. Unstructured and hybrid retrieval remain useful for thematic recall, exploratory search, and broad contextual grounding. This paper targets workloads where correctness depends on stable records: exact facts, current state, updates and deletions, aggregation, relations, negative queries, and explicit unknowns.

Most external memory systems store text, embed it, and retrieve the most similar chunks at query time [3–5]. That loop works well for questions that ask for theme and context, but practical memory is often used as an input to action. Those workloads ask questions such as:

1. **Thematic recall** — “What were we discussing about X?”
2. **Single-fact lookup** — “What timeout did we set?” or “Which database did we choose?”
3. **Aggregation** — “List all APIs that failed” or “How many times did latency concerns appear?”
4. **State queries** — “What is the current status?” where overwrites and updates matter.
5. **Relational queries** — “Which users reported errors after version X?” which requires linking across items.
6. **Negative and exclusion queries** — “Which decisions have not been revisited?” where absence is meaningful.

Semantic similarity is a strong match for (1). For (2) through (6), it is only a heuristic: it retrieves documents that are *about* a topic, not records that are guaranteed to *contain a specific fact*. Embeddings capture topic and context, but they are not predicate evaluators, and high similarity is not evidence of fact presence [4, 6].

The precision requirements of memory are also higher than those of search. Search tolerates near-misses; memory does not. If memory returns the wrong value, omits a condition, or surfaces outdated state, downstream reasoning is corrupted. Long context windows only delay this failure mode: facts remain buried among irrelevant text, the model must re-parse them on every read, and interpretive errors can recur across prompts [7–9].

Many memory questions are computational rather than retrieval-only. They require counting, filtering, grouping, deduplication, ordering, comparison, joins, and exclusion. Embedding retrieval returns a ranked subset, not an exhaustive, constraint-satisfying result. Over time this

matters because memory is reused: a recall error can become an input to future reasoning and future memory writes. Weak recall is not just a one-off failure; it is a mechanism of long-term drift.

If AI memory must support exact facts, aggregation, state tracking, and relational queries, then semantic similarity over unstructured text is insufficient by construction.

1.2 Summarisation and embedding-based recall introduce unavoidable loss

External memory systems differ in how much they compress what they store. Some document-store RAG systems retain source text in full; others summarise, chunk, embed, or selectively retain details to reduce cost and latency [3, 4]. These choices create two related failure modes. If the original text is retained, the fact may still be present but unreachable because retrieval is not predicate evaluation. If the text is compressed, the fact may be lost before any future query is known.

A compact way to state the compression problem is to treat memory writing as a transformation. Let X be the original text or history, let $Z = g(X)$ be a compressed representation, and let A denote the answer to a future factual query. By the data processing inequality [10–12]:

$$I(A; Z) \leq I(A; X).$$

Compression can be useful, but it cannot increase the information available about future answers.

This is especially damaging for memory because future queries are not known at write time. Summaries preserve salience and narrative coherence, while embeddings preserve neighbourhood structure in a vector space [4, 6, 13, 14]. Memory queries often depend on low-salience details: exact values, rejected options, negation, units, timestamps, and null-vs-value distinctions. Once such details are summarised away, merged into a vector neighbourhood, or dropped as “irrelevant”, they cannot be reliably reconstructed at read time.

Unstructured memory therefore fails in two ways. It may contain the right evidence but fail to retrieve it, or it may have compressed the evidence away entirely. Both failure modes push the model toward inference and plausible completion rather than evidence-grounded recall [15]. The next section explains why this remains true even when retrieval quality improves.

2 Why unstructured memory fails at factual recall

2.1 Implicit relevance vs explicit relevance

Section 1.1 argues that factual memory requires lookup semantics: exactness, completeness, temporal consistency, and meaningful absence. Unstructured memory approximates this with chunks, similarity, reranking, and prompting [4, 16]. The central issue is that relevance remains implicit. The system never explicitly records which facts are addressable, which values must be preserved exactly, or which unknowns must not be inferred.

This produces three predictable failures:

- **Precision failure** — retrieved text is related but does not contain the exact fact.
- **Omission failure** — low-salience details are compressed away or never rank highly enough.
- **Substitution failure** — the system infers a plausible value when evidence is missing [15].

Substitution is especially damaging because guessed values often look syntactically valid and semantically reasonable; once treated as truth, they become seeds for downstream memory corruption.

Absence is the sharpest case. A conversation may explicitly say “we never settled on a timeout” or “that option was ruled out”, but answering an absence query requires more than retrieving one related passage. The system must know whether a value was explicitly unknown, explicitly rejected, or simply never mentioned, and it must be confident that no later statement contradicted it. Semantic retrieval has no native mechanism for exhaustive non-existence checks across a corpus; it can only rank candidates. In the absence of a definitive hit, the model is pushed toward inference rather than evidence.

Factual recall therefore fails because relevance is implicit, approximate, and recomputed at read time, while factual memory requires explicit, stable, and addressable representations.

2.2 Limits of semantic similarity for precise questions

Similarity optimises for “aboutness”, while factual memory needs correctness under constraints. The mismatch is persistent for two reasons. First, similarity is continuous while correctness is discrete: a ranked list has no natural cutoff where relatedness becomes truth. Second, precise questions contain executable operators: equality, inequalities, temporal predicates, conjunctions, exclusions, and joins. Embedding spaces do not execute these operators. Even when retrieval lands on the right chunk, the final answer still depends on prompt-dependent interpretation rather than stable lookup semantics.

Semantic similarity is therefore well suited for exploratory recall and contextual grounding, but it is not enough for the exactness, determinism, and completeness required by factual memory.

2.3 Why scale, reranking, or hybrid retrieval do not fix factual recall

It is tempting to treat unstructured memory as an engineering problem: more chunks, better rerankers, longer context, hybrids with keyword search. These techniques reduce symptoms, but they do not change the semantics [17]. They still infer relevance at read time and still require re-parsing and inference to produce answers.

Table 1: Why common retrieval upgrades improve coverage but not factual guarantees.

Technique	What it improves	What it cannot guarantee
Scaling (more chunks, bigger stores)	Higher chance that some relevant text exists	Exactness, completeness, stable cutoffs for correctness
Reranking	Better ordering of a similarity-driven candidate set	Predicate satisfaction, missing fact detection, determinism
Hybrid retrieval	Broader recall surface via multiple signals	A clear correctness model: multiple signals can agree on the wrong answer, disagree without a resolution rule, or both miss a fact; failure modes multiply rather than simplify
Long-context stuffing	Avoids explicit retrieval by putting more text in context	Structured recall, stability across prompts, absence detection

The practical stress tests follow directly from Section 2.1: can the system represent unknown vs not mentioned, handle absence, and remain deterministic across runs. Unstructured memory systems struggle with these by design, regardless of scale. Improving semantic retrieval does

not transform it into factual memory. To support precise recall, relevance itself must be made explicit rather than inferred.

3 Schema-grounded memory as explicit relevance definition

3.1 Schemas as contracts for what must be remembered

A schema functions as an explicit, enforceable contract that defines which facts must be preserved and which can be safely ignored. This turns memory from a heuristic process into a governed system.

Reliable memory requires a definition of obligation. In practice, a system needs to be able to say:

- what *must* be remembered,
- what *may* be forgotten,
- what must never be approximated or guessed.

Unstructured memory can express an obligation—a summarisation prompt can instruct the model to always retain certain fields—but it cannot enforce it. Compliance depends on the model following the instruction correctly on every write, with no validation gate to catch omissions or approximations. Schemas make obligation structural rather than instructional: a missing required field is a detectable error, not a silent one.

Schemas formalise intent and domain rules at design time. They specify entities, fields, allowed values and constraints, and relations across entities—in the relational database sense: foreign-key style links that make cross-entity queries expressible as joins rather than inference. This is not an implementation detail. It is a declaration of intent: “These facts matter enough to be stored exactly.”

Once a schema exists, relevance becomes inspectable and testable. Completeness can be validated and missing data becomes detectable. Without a schema, loss is silent and errors surface only downstream.

Schemas also define the boundaries of compression. They allow aggressive compression of irrelevant text while guaranteeing zero compression of critical facts. This is a direct response to the loss mechanisms discussed in Section 1.2.

Finally, schemas create stable semantics. A schema does not change with phrasing and does not drift with context. If a required field cannot be extracted or violates constraints, the system can fail loudly, defer action, or request clarification.

By acting as explicit contracts for what must be remembered, schemas transform memory from implicit inference into a governed, verifiable system.

Toy example. Consider a user who says: “We tried Postgres but latency was too high, so we switched to Redis for the session store.” An unstructured system stores this as text and, asked later “What database are we using for the session store?”, must re-read and re-interpret the passage—with no guarantee it will not surface Postgres as a plausible answer. A schema with an entity `ServiceConfig` and required fields `component`, `database`, and `status` forces the write path to extract `{component: "session store", database: "Redis", status: "active"}` and to record Postgres separately as `status: "rejected"`. The read path then executes a direct field lookup: no inference, no re-parsing, and the rejected alternative is retrievable as a distinct, inspectable fact rather than buried in prose.

3.2 Memory as facts, not text

Once relevance is defined by a schema, memory should store atomic facts aligned to that schema rather than unstructured text fragments. This is a representational shift, not an optimisation.

Text entangles facts with narrative. Natural language mixes facts with explanations, hypotheticals, retractions, and tone. When memory stores text, boundaries are ambiguous and interpretation is required on every read.

Facts must be stored as addressable units. A memory fact should be individually identifiable, directly retrievable, and independently verifiable. Schemas define what counts as a fact, how facts are decomposed, and how entities relate. This enables normalisation: deduplication, precise queries, and updates without rewriting entire narratives.

Fact-based memory eliminates repeated inference. Retrieval becomes lookup and computation over values. This yields determinism: the same query yields the same answer, and updates become explicit events rather than implicit reinterpretations. Facts also make absence and missingness first-class, see Section 2.1.

Storing memory as schema-aligned facts rather than text transforms recall from probabilistic inference into deterministic retrieval.

4 Architectural tradeoff: write-path complexity vs read-path simplicity

4.1 Unstructured retrieval (RAG-style) vs structured memory

Schema-grounded memory does not remove cost; it moves it. The term RAG covers many approaches [3], including hybrid retrieval, graph-augmented retrieval, and LLM-mediated database access. Here, when we use RAG as a contrast class, we mean the common memory pattern of storing source text or summaries and retrieving text-like items by semantic or hybrid similarity at query time. Other members of the broader family, including graph-structured and relational variants, are discussed in Sections 4.2 and 4.3.

RAG-style memory keeps writes cheap by storing text and deferring interpretation. Structured memory pays interpretive cost at write time by extracting, validating, and storing facts before they are queried.

Table 2: Two memory architectures optimise for different failure profiles.

Dimension	Unstructured retrieval (RAG-style)	Schema-grounded memory
Write path	Store text, optionally embed	Extract and validate facts, store records
Read path	Retrieve and re-interpret	Query and compute over fields
Primary strength	Thematic recall and context	Exact lookup, state, aggregation, joins
Failure mode	Silent approximation and drift	Explicit missingness and validation failures
Debuggability	Low (latent thresholds)	High (inspectable records and constraints)

A useful way to state the tradeoff is simple: retrieval works well when approximate context is

enough; structured memory is needed when facts are reused, precision matters, and correctness compounds over time.

In RAG-style memory, every read repeats the same work: select candidates, assemble context, infer facts from text, resolve contradictions, and guess missing values. Errors surface late, and the source of the error is ambiguous. In structured memory, extraction and validation happen before persistence, so reads become direct lookup, filtering, joins, and aggregation over explicit fields.

Hybrid retrieval is the realistic baseline, not a weak strawman. Dense plus keyword retrieval, reranking, graph traversal, temporal filtering, query rewriting, and extracted memory items can all improve recall coverage. The limitation remains the same unless facts and operations are explicitly represented: hybrid retrieval improves the chance of surfacing the right chunk, edge, or memory item, but the result still has to be interpreted at query time. It does not by itself provide exactness, completeness guarantees, deterministic aggregation, or reliable state transitions. The gap between hybrid retrieval and schema-grounded memory is therefore narrower in coverage, but still different in kind.

By shifting complexity from the read path to the write path, schema-grounded memory trades ingestion cost for correctness, stability, and long-term reliability.

4.2 Graph RAG: introducing partial structure into retrieval

Graph RAG[18] introduces explicit relationships into retrieval, and in doing so supports the broader thesis of this paper: pure similarity is not enough, and structure helps.

Benefits include:

- improved multi-hop retrieval,
- relationships made explicit,
- reduced ambiguity in navigation,
- constrained traversal over embedding stores.

The limits are equally important. In most implementations, Graph RAG still stores nodes as text or embeddings, still relies on semantic similarity at the leaves, and therefore still lacks field-level completeness guarantees and enforceable correctness criteria. Graphs organise memory, but they do not define factual obligations.

Table 3: Graphs can improve navigation, but they are not a substitute for schema-governed facts.

Aspect	Graph RAG	Schema-grounded facts
What is explicit	Relationships and traversal	Fields, constraints, and obligations
Leaf representation	Often text or embeddings	Normalised records
Correctness model	Emergent and prompt-dependent	Testable and enforceable
Best fit	Multi-hop context retrieval	Deterministic memory queries

Graph RAG improves retrieval navigation, but it does not solve factual recall or compounding error because it does not shift interpretation to the write path.

4.3 Why not a general-purpose relational database?

The progression above—from unstructured RAG through graph-structured retrieval—raises an obvious endpoint question: does a classical relational database, Postgres for instance, with logic to extract structured data from text, already solve the problem? In principle, yes. A relational database provides exactly the correctness properties this paper argues memory must have: enforceable schemas, exact field lookup, aggregation, joins, negation queries, and explicit nulls for unknown values. The problem is not with the storage layer. The problem is with the agent-database interface on both the write and read paths.

Write path. A relational table does not populate itself from natural language. Getting agent observations and conversation content into structured records requires a reliable extraction layer that reads text and produces schema-compliant, validated rows. That extraction problem—converting unstructured input into correct structured records—is precisely what this paper addresses. A Postgres schema defines what must be stored; it cannot enforce how faithfully an LLM-driven extraction step fills it. The write-path reliability gap is the same whether the backing store is a vector database, a document store, or a relational database.

Read path. Agents querying a general relational database must generate SQL from natural language. Text-to-SQL has received substantial research attention, and frontier models perform well on clean, small-schema academic benchmarks. However, performance degrades sharply on realistic settings: the Spider 2.0 benchmark, which uses real enterprise databases with thousands of columns and multi-dialect SQL, finds that even the best agent frameworks solve only around 21% of tasks [19]. Schema linking errors—the model connecting the natural-language term to the wrong column or table—account for the largest share of failures. General-purpose relational schemas are designed for human developers and applications, not for agents. They are typically normalised, multi-table, require joins, and carry institutional naming conventions that agents must infer without context.

Schema co-design. These failure modes point to a design opportunity rather than a fundamental limit. A memory schema co-designed for agent access can be kept flatter, can use explicit unknown fields, and can be organised around the query patterns agents actually issue rather than around normalisation rules for human applications. The schema remains a contract—with all the correctness guarantees that implies—but it is a contract written for its actual consumer. This is the design position taken in this paper: schema-grounded memory that sits between raw unstructured storage and a general-purpose relational database, inheriting correctness from the former and agent-compatibility from the latter.

4.4 Text-to-SQL as constrained retrieval

Once memory is stored as schema-aligned records, retrieval becomes a constrained translation problem rather than open-ended semantic search. A practical interface is text-to-SQL or an equivalent structured query language. The value is not SQL per se—it is boundedness.

Constrained retrieval improves reliability because:

- outputs have strict validity checks, and invalid queries are detectable,
 - partial correctness is measurable, enabling retries and deterministic failure handling,
 - the query language expresses exact matching, joins, aggregations, ordering, and negation,
 - queries can be logged, inspected, audited, and replayed—and crucially, so can their results.
- Unstructured retrieval logs are natural-language request and response pairs: verifying correctness after the fact requires re-reading and re-interpreting both. A structured query log records the exact predicate evaluated, the fields returned, and whether any constraints

were violated. Debugging is mechanical rather than interpretive, and audits can verify whether a specific fact was or was not in memory at a specific point in time.

It also reduces cognitive load on the model. Instead of rereading long text and re-inferring facts, the model operates on compact, normalised records, which reduces prompt sensitivity and variance.

By turning retrieval into a constrained translation problem, schema-grounded memory simplifies the read path while increasing precision and predictability.

The decision-latency and token-cost implications of this tradeoff are discussed after the write-path design in Section 6.4.

5 Empirical limits of single-pass structured output

Object extraction is difficult because it bundles several coupled decisions into one commit: detecting whether an object exists, deciding which fields are present or applicable, extracting values with the right types and normalisation, and checking consistency across fields. Field dependencies make these decisions cascade; a date may depend on event detection, a numeric value may depend on units and scope, and optional fields may depend on context. For memory, partial correctness is often insufficient: one wrong required field can make a record unusable, while silent correction hides uncertainty and introduces untracked edits. This is why single-pass structured output must be evaluated at the record and output level, not only at the field level.

5.1 Field accuracy vs object accuracy

Single-pass structured output often looks strong on surface metrics [20]. Many extracted fields are correct. The problem is that memory does not consume fields independently—it consumes records. A record with one wrong required field is operationally wrong, even if most fields are correct.

This gap has a precise model, grounded in how autoregressive generation actually works. Transformer-based LLMs factorize the joint distribution of a generated sequence using the chain rule of probability. For a record with m required fields $V_1, \dots, V_m$ extracted in order, the probability that the full record is correct decomposes exactly as:

$$P(\text{record correct}) = \prod_{i=1}^m P(V_i \text{ correct} \mid V_1, \dots, V_{i-1} \text{ all correct}).$$

Let $q_i = P(V_i \text{ correct} \mid V_1, \dots, V_{i-1} \text{ all correct})$ denote the per-field accuracy conditioned on all prior fields being correct—which is precisely the scenario in which the KV cache contains only valid context. Then:

$$P(\text{record correct}) = \prod_{i=1}^m q_i.$$

This is the natural decomposition: q_i is the accuracy of extracting field i when the model has a clean, correct conditioning context. If fields were truly independent and each had the same accuracy p , this reduces to p^m —but that is a special case, not the general rule.

The chain rule decomposition should be read narrowly. The product $\prod_i q_i$ is not an additional bound caused by off-path errors; it is exactly the probability of the fully-correct path—all fields correct in sequence, with every field conditioned on a clean prefix. If any early field V_j is extracted incorrectly, the event “record correct” has already failed. Let $r_i = P(V_i \text{ correct} \mid \text{some prior field wrong})$. The role of r_i is therefore not to enter the all-correct probability

above, but to describe what happens to partially correct outputs after a corrupted prefix: a wrong generated value can bias the KV cache and shift later conditional distributions toward inconsistent completions, often making downstream errors mutually reinforcing. The single-pass weakness has two parts: the probability of an entirely correct record decays multiplicatively as $\prod_i q_i$, and errors off that all-correct path tend to propagate rather than remain isolated. As a concrete illustration, if $q_i = 0.97$ for all fields and $m = 20$, the probability of a fully correct single-pass record is $0.97^{20} \approx 0.54$. This explains why even high per-field accuracy produces unacceptably low record-level accuracy at realistic schema sizes.

In a one-shot setting this collapse is unavoidable: there is no mechanism to interrupt the chain, catch an early error before it corrupts downstream conditioning, or retry a single field in isolation.

Iteration changes this by breaking the dependency chain [21]. In decomposed extraction, field i is only placed into the conditioning context after passing a validation gate, so errors caught by validation do not create a corrupted prefix for downstream fields; downstream prompts condition on validated, correct context. If field i fails validation and is retried k_i times, the probability that at least one attempt succeeds is:

$$p_i^{(k_i)} = 1 - (1 - q_i)^{k_i}.$$

Since each retry operates under clean context for errors caught by validation (prior fields already validated), retries are independent conditional on that validated prefix, and the record-level probability becomes:

$$\prod_{i=1}^m p_i^{(k_i)}.$$

Using the same illustration: if $q = 0.97$ and $m = 20$, two attempts per field yields $(1 - (1 - 0.97)^2)^{20} \approx 0.98$, compared to the single-pass all-correct probability of 0.54. In practice, retries are applied only to fields that fail validation, concentrating additional work on uncertain fields rather than regenerating the whole object.

The failure is not just a lower number. It is a different error surface. Object-level failures are often driven by a small set of recurring issues:

- missing optional fields that become required by downstream logic,
- incorrect null-vs-value decisions,
- unit and format mistakes that pass superficial checks,
- misassigned boundaries (values attributed to the wrong entity or field).

Single-pass extraction maximises joint error probability because it forces all decisions to be made at once, with no opportunity for validation feedback or local correction. For memory, storing incorrect facts is often worse than storing none, because incorrect records become confident, reusable inputs.

The persistent gap between field-level and object-level accuracy shows that single-pass structured output is insufficiently reliable for memory systems, even when individual field extraction appears highly accurate.

5.2 Compounding loss and memory corruption

Memory correctness is a joint property, not a local one. Workflows rarely depend on a single fact. They depend on multiple facts retrieved together, consistency across those facts, and correctness across multiple steps.

If a workflow depends on n facts, each correct with probability p , then the probability that all required facts are correct is roughly p^n . If a fact is interpreted k times, the probability it remains correct is p^k . When both effects apply—multiple facts and repeated interpretation—correctness degrades multiplicatively. [22]

Memory corruption often emerges from plausible inaccuracies. [15] Substituted numeric values, inferred defaults, and missing qualifiers can remain internally consistent and pass unnoticed. Aggregation magnifies the cost of small errors: planning, summarisation, comparison, and constraint checking are sensitive to any incorrect input, and one wrong fact can derail an entire chain.

Because most memory errors do not crash the system, they accumulate silently. This is the same drift mechanism introduced in Sections 1.1 and 1.2, now applied to multi-fact workflows.

When memory-backed workflows depend on multiple facts and repeated interpretation, even high per-step accuracy is insufficient, as correctness degrades multiplicatively with workflow length and factual dependency.

6 Iterative, schema-aware extraction

6.1 Overall architecture: object detection, field detection, field value extraction

Section 5 explains why one-shot structured output is brittle at the record level. The remedy is to turn extraction from a joint prediction into a controlled sequence of narrower decisions, each with validation gates and local retries [22–25].

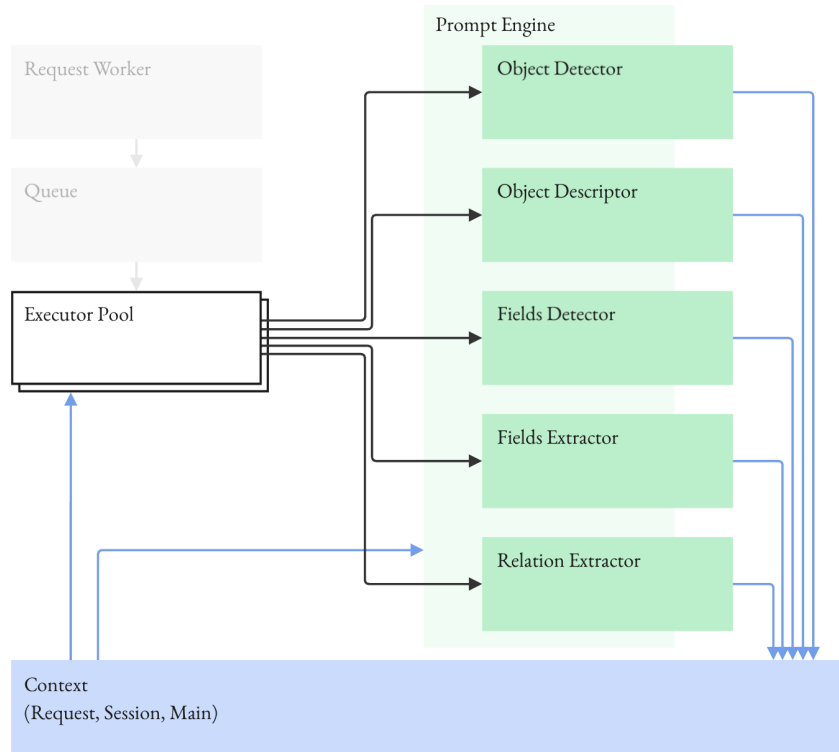


Figure 1: Iterative extraction pipeline: staged decisions with validation gates and local retries.

A minimal decomposition has three stages:

1. **Object detection** — does an object of this schema exist at all?

2. **Field detection** — which fields are present or applicable, based on evidence?
3. **Field value extraction** — what are the values for the detected fields, under type and normalisation constraints?

The benefit is not only modularity: each stage turns a broad schema-generation task into a narrower decision problem, such as presence detection, span extraction, or constrained value normalisation, where modern models are typically more reliable.

This staged framing is aligned with MuSEE [24], which also decomposes structured entity extraction into multiple stages and proposes entity-centric evaluation. Our focus is different: reliable long-term memory. We treat schemas as memory contracts, place validation and local retries on the write path, maintain request, session, and main contexts, and evaluate extraction, update and diff correctness, and end-to-end read accuracy.

Decomposition changes the error surface. Object detection errors do not automatically corrupt field-value extraction; field detection errors limit scope without forcing incorrect values for detected fields; and value errors can be retried locally without regenerating the whole object. Validation gates make this possible by checking object evidence, allowed field combinations, types, ranges, formats, normalisation, and explicit unknowns.

Single-pass extraction still has one real advantage: all fields are generated in one context, so the model can sometimes use cross-field coherence to avoid contradictions. That advantage is strongest for simple, flat schemas. It weakens as schemas grow, as records span multiple objects, or as the task requires intent-sensitive operations such as add, update, clear, delete, and rename. In those settings, conditioning on previously *generated* values is risky because an early mistake can become context for later fields. Decomposed extraction instead conditions later prompts on previously *validated* decisions. A further complication is that single-pass structured output often relies on grammar-constrained decoding, which can enforce format while still distorting the model’s semantic distribution [26].

Validators turn extraction errors into retrievable events rather than silent commits. When validation fails, the system can retry the failing stage, ask for clarification, or store an explicit unknown. Iteration improves accuracy without expanding scope: it refines only what failed and avoids rewriting correct parts. Schema awareness is what makes this architecture possible. Without a schema, there is no stable field set, no validator target, and no stage-specific success criterion.

6.2 Prompt engine role

Decomposition requires a control plane. A schema-aware prompt engine turns prompts into stateful control logic rather than static instructions. In single-pass extraction, the prompt must present all uncertainty at once, and the model must infer structure and content simultaneously. In iterative extraction, prompts evolve based on extracted state: resolved facts are treated as fixed, and unresolved questions are isolated.

Conditioning reduces ambiguity at each step. After object detection, prompts no longer ask whether an object exists, they focus on details. After field detection, prompts do not search the entire schema space, they target a known finite set. After partial extraction, prompts can reference confirmed facts and avoid re-resolving identity.

A concrete example illustrates the difference:

- Monolithic: “Extract a person object from the paragraph.”
- Stateful: “A person has been identified as Claude Shannon, the scientist. Based on the schema, which fields are explicitly mentioned for this person in the paragraph?”

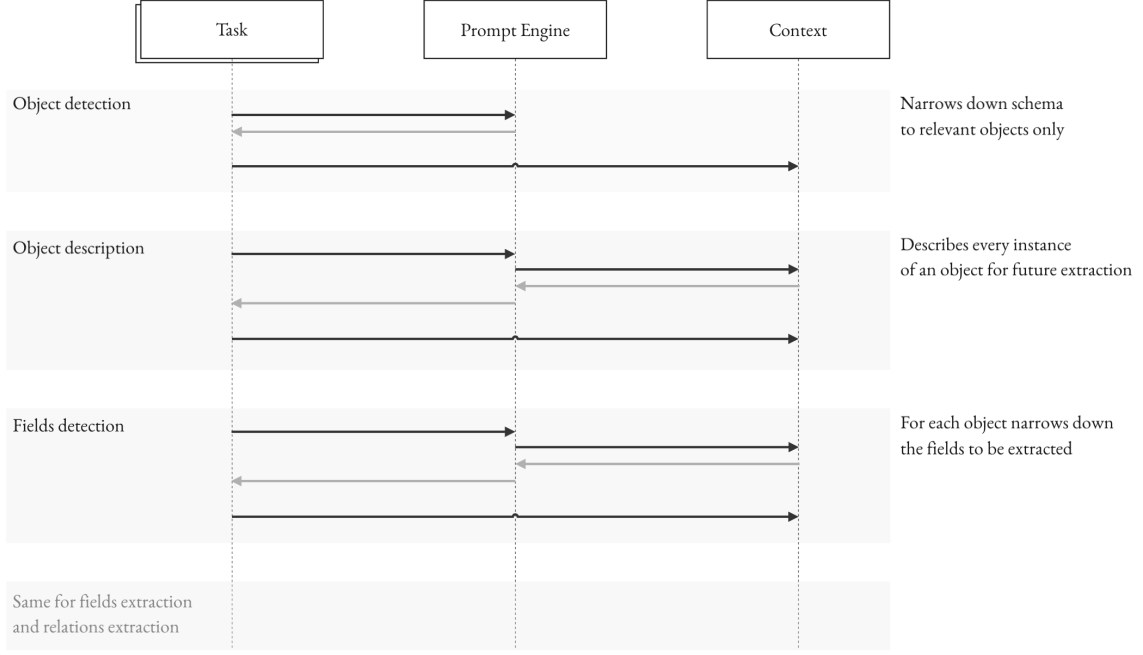


Figure 2: Prompt engine control flow: prompts evolve from extracted state, and validation feedback targets local retries rather than full regeneration.

Field-level prompts then enable precision extraction by reinforcing field definitions and type constraints. Schema awareness also enables negative constraints, which operationalises the absence distinction from Section 2.1 in the write path (e.g., “if no expiry date is mentioned, set `expiry_date` to null; do not infer or approximate one”).

Finally, validation feedback loops turn generation into guided correction. When validation fails (invalid formats, conflicting values, missing required fields), the prompt engine can incorporate that feedback and request targeted correction rather than full regeneration.

By making prompts stateful, schema-aware, and conditioned on prior validated decisions, the prompt engine converts extraction from a single inference problem into a controlled, high-precision process.

6.3 Memory contexts: request, session, and main

Iterative extraction is naturally multi-worker: different components detect candidate objects, infer applicable fields, and extract values under constraints. [22] To coordinate these steps without turning every prompt into a global reasoning problem, we separate *memory context* into three scopes with different lifetimes and responsibilities [1].

Request memory context. The request context is ephemeral and shared by all workers handling a single user request or ingestion job. It contains the chunked inputs, intermediate hypotheses (candidate objects, tentative links), validation outcomes, and any clarifications produced during the write path. The prompt engine primarily conditions on this context to make each next prompt maximally specific: workers can reference what has already been detected, which fields have been confirmed, and which validation failures must be corrected. Request context may read from broader contexts for grounding, but it is optimised for fast, local decisions.

Session memory context. Many objects are not fully defined within one request [9]. Information may arrive across turns in a chat, across pages in a document, or be fragmented by chunking before the write path. The session context is a medium-lived workspace where the system assembles partial objects over a session, resolves identity incrementally, and tracks what changed within the session. It can ground itself in the main memory when prior values exist, but its focus is the session-local delta: the subset of entities touched in the session, their evolving field values, and unresolved slots that are expected to be filled by subsequent requests.

Main memory context. The main context is the durable, versioned store. It contains the current best-known values, the evolution of those values over time, and explicit provenance: links from records to the sessions (and, by extension, actors and sources) from which facts were derived [27]. This makes data lineage a first-class property of memory rather than an afterthought, and it enables conflict handling and diff computation as explicit operations on records.

Merge and separation of concerns. These contexts partition the write path into narrower tasks:

- request-level detection and extraction, optimised for precision under tight context,
- session-level object assembly, optimised for continuity across requests and chunk boundaries,
- main-memory persistence and diff calculation, optimised for long-term correctness, history, and lineage.

They are not isolated. Knowing the main-memory state improves request-level extraction (disambiguation, normalisation, constraint checking), and session-level hypotheses can guide what the request-level workers search for next. The separation is therefore about *where decisions are finalised*, not about forbidding information flow.

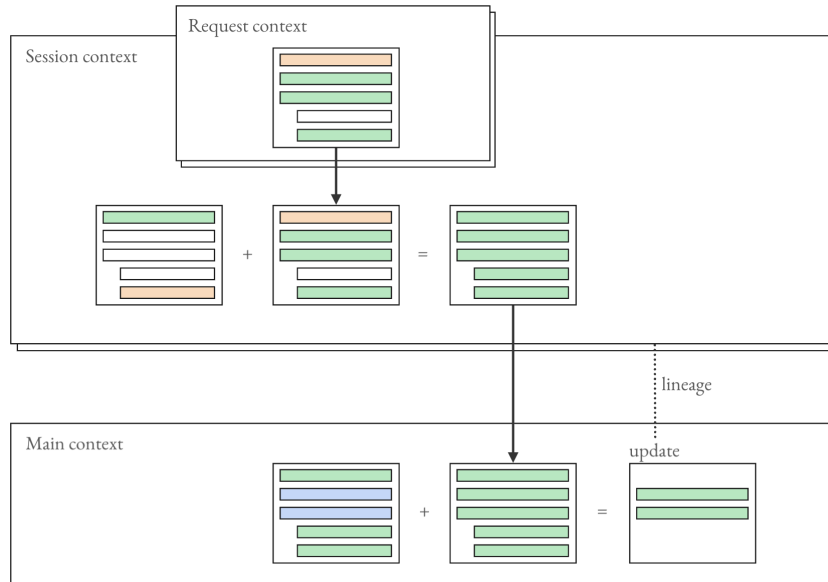


Figure 3: Three memory contexts and their merge flow: request context coordinates workers within a single write path, session context assembles partial objects across requests, and main memory persists versioned records with lineage.

6.4 Real-workflow decision latency and token consumption

The preceding sections describe why `xmemory` uses a more complex write path: it detects objects, detects fields, extracts values, validates intermediate results, retries locally, and manages request, session, and main memory contexts. That extra write-path work has an operational payoff: schema-grounded memory can reduce both token consumption and real-workflow decision latency on read-heavy workloads.

The key distinction is that retrieval latency is not the same as decision latency. A text-based memory layer may search, rank, and return candidate memories quickly. A memory-grounded workflow must then pass those candidates to an LLM, pay model prefill and attention costs over the injected context, and ask the model to filter distractors, resolve contradictions, and infer the answer. Even if candidate retrieval itself is fast, for example sub-200ms in an optimised path, that figure covers only search and ranking; it does not include downstream LLM processing, answer synthesis, or tool orchestration. This is why a system can have low retrieval latency while still producing high end-to-end workflow latency if it returns broad context rather than answer-shaped data.

A vector or hybrid RAG pipeline typically injects several retrieved passages into the model context and asks the model to re-interpret them [3]. Longer contexts increase latency and cost, and accuracy can degrade when relevant information is buried among distractors [7, 28]. Structured retrieval returns only the requested fields or computed results. A query asking for the current database choice for a component returns a compact record, not several paragraphs of surrounding narrative. An aggregation query returns a count, not every source passage from which the count might be inferred. Relational queries that require multiple retrieval rounds in a RAG pipeline can often be expressed as one join over records.

This matters because context is a finite system resource. Every read that returns a precise value rather than a bundle of prose saves tokens that do not need to be parsed, attended to, normalised, or discarded. The advantage compounds in agentic workloads where reads are more frequent than writes. Structure therefore improves three read-path properties at once: correctness, because queries run over verified records; reproducibility, because the same query over the same store returns the same result; and efficiency, because retrieved results are answer-shaped rather than prose-shaped.

For token consumption, we use symbolic token units for proportional comparison rather than measured billing tokens. Let R be the number of reads per write, let w_x and r_x be the write and read token costs for `xmemory`, and let w_t and r_t be the corresponding costs for a text-based memory system. The per-cycle costs are:

$$C_x = w_x + Rr_x, \quad C_t = w_t + Rr_t.$$

The relative token cost of the text-based system compared with `xmemory` is:

$$S = \frac{C_t}{C_x} = \frac{w_t + Rr_t}{w_x + Rr_x}.$$

Using conservative symbolic assumptions, with $R = 10$ reads per write, $w_x = 10$, $r_x = 1$, $w_t = 3$, and $r_t = 6$, we get:

$$S = \frac{3 + 10 \cdot 6}{10 + 10 \cdot 1} = \frac{63}{20} \approx 3.15.$$

Under these assumptions, the text-based system consumes about 3.15 times as many LLM tokens per write-read cycle, while `xmemory` uses slightly less than one third of the tokens. The

write-side assumption $w_t = 3$ is also deliberately favourable to text-based systems. In the third-party systems evaluated in Section 8.3, writes often triggered asynchronous processing that had to finish before comparable reads were possible. This suggests that the effective write path may involve more than a simple append, even when the write API appears lightweight. Because $r_t = 6$ is conservative for text-based reads that often retrieve and synthesise multiple passages, and because $w_t = 3$ may also understate effective write-side work, this should be read as an engineering estimate, not a benchmark result. Latency is not perfectly proportional to token count, but fewer retrieved tokens generally reduce model-side prefill and attention work; the same mechanism that produces token savings is therefore also the mechanism behind the decision-latency advantage.

7 Schema lifecycle: bootstrapping and evolution

Section 6 treats the schema as an input to the write path, but in practice the schema is a product decision. It defines which facts the system is obligated to store, validate, and later answer with high confidence. A reliable deployment therefore needs a schema lifecycle: bootstrapping, validation against intended questions, and controlled evolution.

7.1 Bootstrapping the schema

Most systems start from one of two sources. Organisations may already have governed schemas in CRMs, ERPs, ticketing systems, or warehouses; in that case the memory schema should usually import a narrow projection of the existing system of record. In personal workflows, greenfield products, or rapidly changing domains, the schema may be authored directly in YAML, JSON Schema, or a UI [29, 30]. In both cases, starting small is usually better: store the facts needed for high-value questions, and represent missing values as explicit unknowns rather than speculative fields.

7.2 Agent-assisted schema design from intended questions

Users rarely begin by specifying schemas. They specify questions they expect the system to answer later. A practical design loop is therefore: elicit representative questions; propose entities, fields, and relations that make them answerable as structured queries; test whether each question can be translated against the proposed schema; and iterate until the schema is the minimal contract that supports the intended workload [12, 21, 31]. This connects schema design directly to the query taxonomy in Section 1.1: if users need aggregation, joins, state transitions, or exclusions, the schema must represent those concepts explicitly.

7.3 Schema evolution from observed usage

Once deployed, real usage reveals schema gaps. Repeated requests for missing attributes, recurring ambiguity, implicit relations between entities, and repeated validation failures can all indicate that the contract should change. Schema evolution should be expressed as versioned migrations [32, 33]. A migration updates the schema, prompts, validators, and backfill logic, while marking irrecoverable values as explicit unknowns. This keeps schema improvement auditable rather than turning it into prompt drift.

Closing the loop between observed questions, schema-change proposals, and migrations creates a path toward self-evolving structured memory. The point is not uncontrolled automation; it is that schema-bound memory can improve through concrete, testable changes to a contract.

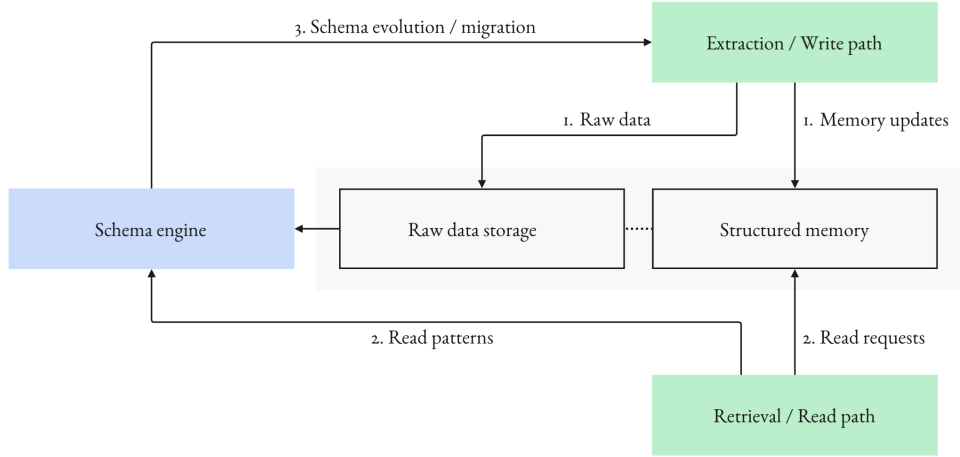


Figure 4: Schema evolution loop: observed questions and failures drive migration proposals; migrations update schema, prompts, and validators, and backfill where possible to improve long-term memory quality.

With this lifecycle in mind, we can now evaluate the central claim of this paper: memory quality improves as structure increases.

8 Evaluation: memory quality improves with structure

8.1 Benchmarks

Memory is an end-to-end system [1, 5, 8, 9]. The evaluation therefore measures three layers together: extraction, storage semantics, and retrieval. The three experiments in this paper serve different roles. The insurance-claims benchmark tests write-path structured extraction. The end-to-end memory benchmark tests updates, deletions, relations, state, aggregation, and negative queries. The Splitwise application benchmark tests whether a system can accumulate structured events expressed in natural language and answer downstream computed questions.

In principle, an LLM with unbounded compute, unlimited context, and enough iterative passes could infer many facts from a corpus. The practical issue is that this regime is expensive and unstable: inference remains generative, outcomes can vary across runs, and repeated long-context passes increase cost. The benchmark question here is operational: which architecture gives reliable factual memory under realistic write and read patterns?

Table 4: Three layers of memory that must be evaluated together.

Layer	What is measured	Typical metric
Extraction (write)	Can the system write correct records from text?	Object accuracy, validation recovery
Storage semantics	Can the system represent updates and unknowns explicitly?	CRUD correctness, versioning tests
Retrieval (read)	Can the system answer strict queries deterministically?	Exact match, completeness, abstention

This framing is intentionally stricter than common practice. Storage semantics is harder to compare directly because most memory benchmarks evaluate retrieval quality rather than

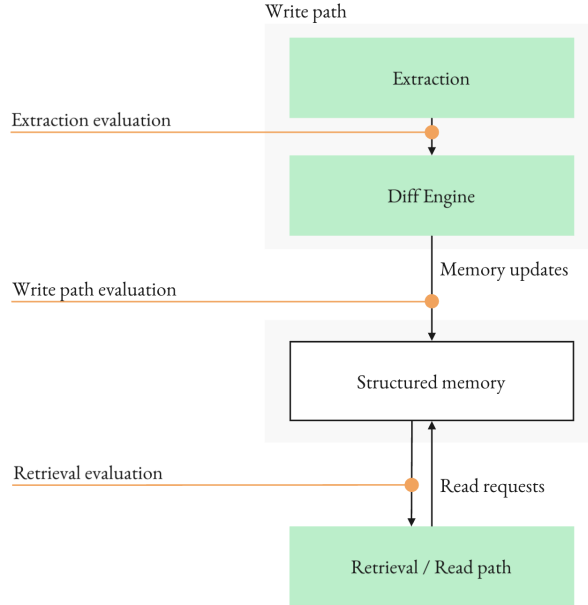


Figure 5: Measurement points in a schema-grounded memory system: write-path extraction, update and diff application, and read-path query answering.

whether the memory layer behaves like a system of record. Extraction alone does not prove memory quality, and retrieval alone hides write-path corruption. Datasets and measurement methodology is available at <https://github.com/xmemory-ai/datasets>.

8.2 Extraction results and comparison

Extraction reporting should make object-level usability the primary target. In addition to field-level metrics, report object accuracy under strict criteria, error categories (missing field, wrong value, null-vs-value mistakes, normalisation errors), and the behaviour of validation loops (retry rate and convergence). Because the proposed architecture uses local retries, convergence curves become an informative signal of system health.

We evaluate structured extraction on the modified *insurance claims* dataset from Cleanlab’s structured output benchmark [34]. Cleaned version of this dataset with schema is available at <https://github.com/xmemory-ai/datasets/tree/main/extraction>.

Dataset. Each example consists of an insurance claim document paired with a ground-truth structured record. The schema covers four areas:

- **Basic claim** — claim ID, claim report date, and related header fields.
- **Insurance policy information** — policy number, coverage type, and associated policy metadata.
- **List of insured objects** — one or more items covered under the policy, each described by its own sub-schema.
- **Incident information** — date, location, description, and other details of the reported incident.

Ground-truth objects following this schema were extracted from the claim documents and serve as the reference for evaluation.

Metrics. We report three complementary metrics at increasing levels of strictness:

- **Field-level precision / recall / F1** — computed over individual scalar fields of the extracted objects.
- **Object-level accuracy** — the proportion of individual objects that were extracted with every field exactly matching the ground truth.
- **Output-level accuracy** — the proportion of claims for which *all* objects were fully correct, i.e. the entire structured output for a claim matches the ground truth.

The systems tested include a few frontier-model extraction baselines (Anthropic Sonnet 4.6, Anthropic Opus 4.7, OpenAI GPT-5.4, OpenAI GPT-5.4 with advanced reasoning, OpenAI GPT-5.5, OpenAI GPT-5.5 with high reasoning effort, and Gemini 3.1 Pro preview), plus two xmemory deep-mode configurations: a single-model extraction pipeline and a variant that adds an LLM judge in the loop. All systems are evaluated on the same dataset, label set, and run protocol to keep comparisons directly interpretable.

Table 5: Field-level precision, recall, and F1 (10 runs, mean $\pm$ stddev)

System	Precision	Recall	F1
Anthropic Sonnet 4.6	95.22% $\pm$ 0.4072%	97.62% $\pm$ 0.366%	96.40% $\pm$ 0.3342%
Anthropic Opus 4.7	96.72% $\pm$ 0.4578%	96.41% $\pm$ 0.2624%	96.56% $\pm$ 0.2745%
OpenAI GPT-5.4	96.22% $\pm$ 0.43%	94.11% $\pm$ 0.79%	95.15% $\pm$ 0.45%
OpenAI GPT-5.4 with advanced reasoning	97.59% $\pm$ 0.31%	94.83% $\pm$ 0.79%	96.19% $\pm$ 0.44%
OpenAI GPT-5.5	97.05% $\pm$ 0.23%	95.00% $\pm$ 0.86%	96.01% $\pm$ 0.47%
OpenAI GPT-5.5 with high reasoning effort	97.18% $\pm$ 0.36%	95.60% $\pm$ 0.44%	96.39% $\pm$ 0.34%
Gemini 3.1 Pro preview	97.60% $\pm$ 0.3846%	96.74% $\pm$ 0.2002%	97.17% $\pm$ 0.25%
xmemory	96.72% $\pm$ 0.55%	97.21% $\pm$ 0.16%	96.97% $\pm$ 0.31%
xmemory, llm-judge-in-the-loop	97.39% $\pm$ 0.4372%	97.67% $\pm$ 0.6128%	97.53% $\pm$ 0.4578%

Table 6: Object-level accuracy (10 runs, mean $\pm$ stddev)

System	Object-level accuracy
Anthropic Sonnet 4.6	83.56% $\pm$ 1.395%
Anthropic Opus 4.7	83.31% $\pm$ 1.6%
OpenAI GPT-5.4	79.24% $\pm$ 2.16%
OpenAI GPT-5.4 with advanced reasoning	81.69% $\pm$ 1.51%
OpenAI GPT-5.5	81.02% $\pm$ 1.70%
OpenAI GPT-5.5 with high reasoning effort	83.98% $\pm$ 1.29%
Gemini 3.1 Pro preview	89.24% $\pm$ 0.572%
xmemory	86.61% $\pm$ 0.78%
xmemory, llm-judge-in-the-loop	90.42% $\pm$ 1.697%

Table 7: Output accuracy (10 runs, mean $\pm$ stddev)

System	Output accuracy
Anthropic Sonnet 4.6	42.00% $\pm$ 4.216%
Anthropic Opus 4.7	42.67% $\pm$ 4.661%
OpenAI GPT-5.4	34.00% $\pm$ 7.00%
OpenAI GPT-5.4 with advanced reasoning	34.00% $\pm$ 5.16%
OpenAI GPT-5.5	38.00% $\pm$ 5.71%
OpenAI GPT-5.5 with high reasoning effort	44.00% $\pm$ 4.39%
Gemini 3.1 Pro preview	61.67% $\pm$ 3.6%
xmemory	50.00% $\pm$ 3.85%
xmemory, llm-judge-in-the-loop	62.67% $\pm$ 4.919%

The main extraction result is not just that field-level scores are high. The larger gap appears at object and output level, where memory usability depends on all required values being correct simultaneously. This is the regime where decomposed extraction with validation provides the largest advantage.

A discussion of why this evaluation protocol is separated from broader memory benchmarks such as LoCoMo and LongMemEval is provided in Appendix A.2.

8.3 Tests of memory functions and comparison with other memory systems

Dataset. The end-to-end evaluation spans four independently constructed datasets, each modelling a distinct real-world domain: *corporate* (employee records and company relations), *education* (students, courses, and academic programmes), *medical* (patients, diagnoses, and treatment assignments), and *finance* (clients, accounts, and transaction records). Each dataset has its own entity population, its own timeline of write operations, and is evaluated in a fully independent run, so there is no shared state or cross-domain contamination between domains. All four datasets are built around the same underlying schema structure—a few entity types and relation types—adapted to the vocabulary of each domain. In the corporate sub-dataset, for example, the three entity types are *Employee* (fields: full name, job title, department), *Company* (company name, industry), and *Project* (project name, start date, end date, status), with relation types *EmploymentRelation* and *ProjectAssignment*. This schema structure is representative of record-keeping scenarios across all four domains: facts about people, organisations, and their relationships must be stored, updated, and queried reliably over time, and it is exactly the kind of schema discussed in Section 3.1 as a contract that defines what must be remembered. Precision, recall, and F1 are computed by pooling true positives, false positives, and false negatives across all four sub-datasets (micro-averaging), so that each individual fact contributes equally to the aggregate numbers reported in Table 8.

The scenario executes a series of write operations that cover the full range of memory mutations identified in Section 3.2: initial fact ingestion, field-level updates (an employee promotion changing a job title), field clearance (a department membership explicitly set to unknown), relation changes (an employee switching employers), relation deletion (an employee removed from a project), entity rename (a company rebranding from Initech to Inirode, requiring

propagation to all linked employees), and cascading entity deletion (the company dissolving, implicitly terminating all employment and assignment relations). This mutation taxonomy is precisely what text-based memory cannot represent explicitly: as discussed in Section 2.1, state transitions and explicit absences are silent in unstructured systems.

Read queries span the taxonomy from Section 1.1: single-fact lookups (current job title, assigned project), relational queries requiring joins across entity types (which company an employee works at), aggregation queries (how many active projects existed in a given year), temporal ordering queries (which of two projects started earlier), and negative exclusion queries (which project did not yet exist before a given date; who is currently unemployed after a company dissolution). A single query can belong to multiple categories simultaneously; for example, “which company does Alice Smith work at now?” is both a relational query and a state query. Crucially, several queries are asked at multiple points in the scenario—after each relevant mutation—to verify that memory reflects the latest state rather than an earlier one. This directly tests the compounding correctness property from Section 5.2: a system that fails to propagate a field update or a deletion will answer a later state query incorrectly even if the original fact was stored correctly.

The dataset is specifically designed to expose the failure modes of similarity-based retrieval discussed in Sections 1.1 and 2.3. Answering “which company does Alice Smith work at now?” correctly requires tracking two employer changes and a company rename across six write operations—the correct answer changes three times over the course of the scenario. Answering “which active companies are on the market?” after the dissolution requires applying negative exclusion over an entity whose existence was previously asserted. Entity rename propagation tests whether relations are stored as normalised records or merely as co-occurrence signals buried in text.

We compare representative memory-system designs under the same read-side evaluation protocol. This is a productized-system comparison rather than a component ablation. Most tested third-party systems are better understood as hybrid-RAG or graph-memory systems: they combine semantic retrieval with graph structure, filters, reranking, query rewriting, temporal reasoning, or LLM-based memory extraction. We do not include a hand-built vanilla BM25+dense+rerank baseline; instead, we evaluate documented memory products that developers are likely to use as stronger retrieval baselines. The experiment therefore does not isolate the marginal value of BM25, dense retrieval, graph traversal, or reranking individually. It asks a deployment-oriented question: how does schema-grounded memory compare with strong publicly available memory systems that already combine several hybrid retrieval mechanisms?

Cognee is evaluated with temporal and graph-completion search modes enabled, representing a graph-assisted retrieval design. In its open-source documentation and repository, Cognee is described as a knowledge engine that combines vector search and graph databases, with explicit memory operations such as **remember**, **recall**, and **forget** [35]. Architecturally, we treat Cognee as a hybrid graph-RAG memory baseline: its public docs describe a relational store for documents, chunks, and provenance; a vector store for semantic similarity; and a graph store for entities and relationships. Our initial intent was to test Cognee in its managed version, but due to technical issues we ran it in a self-hosted configuration. Test setup used Cognee Python SDK version 1.0.3 with Anthropic Sonnet 4.6 as a main LLM and OpenAI "openai/text-embedding-3-large" for embeddings generation.

Mem0 is evaluated in both no-graph and graph configurations. Mem0’s docs position it as a universal, self-improving memory layer for LLM applications, with both managed and self-hosted deployment modes and add/search memory APIs for integration into agent workflows [5, 36]. We treat Mem0 no-graph as a semantic-memory baseline: its documented search path uses query processing, vector similarity, filters, and optional reranking. We treat Mem0 graph as the graph-enhanced hybrid variant: Mem0’s public materials describe graph memory as preserving entity relationships on top of vector-based semantic retrieval. In our tests we used managed version of Mem0, waiting for full completion of every write including asynchronous enhancements of memories. This ensures that all the data that is expected by the following read request is available for retrieval.

Supermemory is evaluated in a hybrid setup with a 0.7 decision threshold, representing a setup tuned towards higher precision. Supermemory’s developer docs describe a unified context stack that ingests text/files/chats, builds a semantic memory graph and user profiles, and supports both memory-centric retrieval and RAG-style document search from the same context pool [37]. We therefore treat Supermemory as a production hybrid-RAG memory baseline: its public docs describe hybrid memory-plus-document retrieval, relationship-aware graph memory, reranking, and query rewriting. In our tests we used managed version of Supermemory, waiting for full completion of every write including asynchronous processing of data written.

Zep is a managed context-graph memory platform [38] evaluated via Zep Cloud with the `zep-cloud` Python SDK. The underlying engine is Graphiti, an open-source knowledge graph technology designed for semantic memory retrieval. We treat Zep as a temporal knowledge-graph and hybrid-retrieval baseline: Graphiti’s public docs describe full-text search, similarity search, graph breadth-first search over nodes and edges, and reranking methods including RRF, MMR, and cross-encoders. In our evaluation, we configured Zep to use the `edges` search scope (returning extracted facts rather than raw chunks), the `cross-encoder` reranker for high-precision relevance scoring, and a `search_limit` of 10 to retrieve the most relevant memories before synthesis. Retrieved facts were passed through an LLM call with a unified synthesis prompt to produce final answers, maintaining consistency with the other third-party backends.

xmemory is evaluated in deep mode for extraction which involves enhanced reasoning in the pipeline. Anthropic Sonnet 4.6 was used as the main LLM and Opus 4.6 was used as the judge to provide a feedback signal to the extraction pipeline.

A note on write-path cost is important for interpreting both latency and token-consumption comparisons. Several third-party systems perform asynchronous processing after a write request returns, including memory enhancement, indexing, graph construction, or retrieval-preparation steps. To keep the read-side comparison fair, we waited until this processing completed before issuing the next read. In some runs this required waiting minutes before recently written data became reliably available. This does not let us measure the providers’ internal token spend directly, and the delay may include queueing, batching, indexing, or rate limits as well as LLM calls. It does, however, support treating the write-cost assumption in Section 6.4 as conservative: text-based memory systems may expose a cheap write API, but their effective write path is often not just a simple append.

All the systems are evaluated on the same dataset and with the same LLM judge. Memories retrieved by Cognee, Mem0, Supermemory and Zep were summarized by an LLM call with Anthropic Sonnet 4.6 to produce final answer to the read query. Precision and recall are computed

by analysing false positives and false negatives in the final answer compared to the ground truth. Different facts in single answer are analysed separately. For example, if question *"Which active companies are out there on the market?"* has ground truth answer *"Globex Corp, Initech"* — we count these two companies as two separate facts. If the system returns *"Globex Corp, Inirode"* — we count one true positive (Globex Corp), one false positive (Inirode) and one false negative (Initech). This allows us to compute precision and recall at the fact level even when multiple facts are returned in a single answer.

Table 8: End-to-end performance metrics, LLM judge

Memory System	Precision	Recall	F1 Score
Cognee	86.18%	86.18%	86.18%
Mem0 (no graph)	89.08%	85.48%	87.24%
Mem0 (graph)	88.24%	84.00%	86.07%
Supermemory	80.49%	80.49%	80.49%
Zep	77.44%	83.06%	80.16%
xmemory	99.15%	95.12%	97.10%

The strongest third-party result is Mem0 (no graph) at 87.24% F1, while **xmemory** reaches 97.10% F1. The largest qualitative differences appear in state, aggregation, and negative-exclusion queries, where correctness depends on explicit updates and absence rather than related-context retrieval.

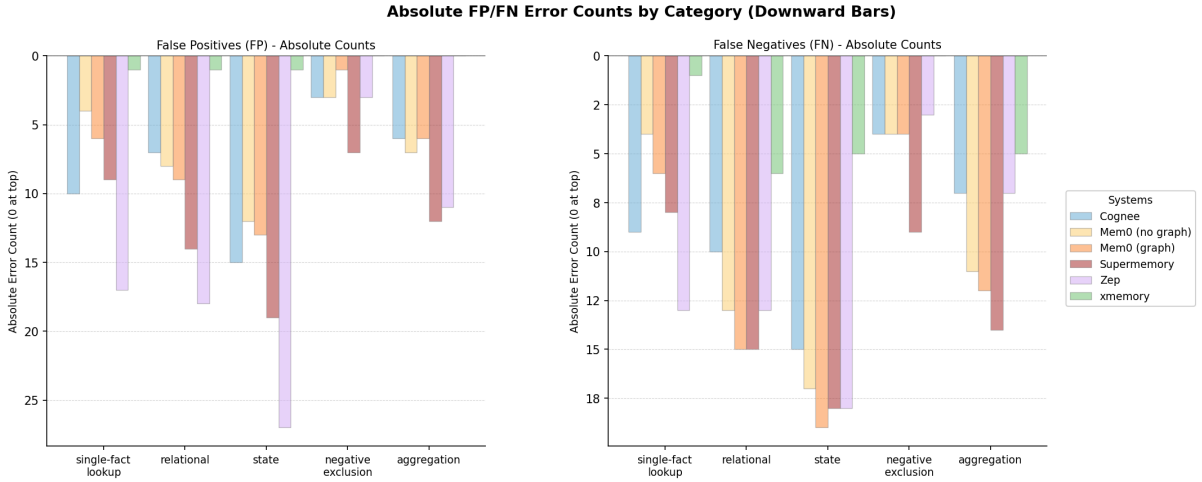


Figure 6: False positives (FP) and false negatives (FN) by query category. Bars extend downward from zero; lower absolute values indicate fewer errors. Counts reflect the number of incorrect facts across all read queries in each category.

All third-party systems without schema constraints accumulate substantial **state** errors, reflecting the difficulty of tracking field updates and entity mutations over time without explicit versioning (Cognee: 15 FP, 15 FN; Mem0 no-graph: 12 FP, 17 FN; Mem0 graph: 13 FP, 19 FN). The near-identical state counts for the two Mem0 variants suggest that adding a graph layer does not meaningfully help with mutation tracking in this setting. Zep’s micro-averaged profile (P=77.44%, R=83.06%) shows consistent FP-heavy errors across categories (single-fact lookup: 17 FP, 13 FN; relational: 18 FP, 13 FN; state: 27 FP, 18 FN; negative exclusion: 3 FP, 3 FN; aggregation: 11 FP, 7 FN), indicating over-generation rather than pure omission. Supermemory shows a more balanced but still error-prone profile (P=80.49%, R=80.49%), with substantial errors

across categories (single-fact lookup: 9 FP, 8 FN; relational: 14 FP, 15 FN; state: 19 FP, 18 FN; aggregation: 12 FP, 14 FN; negative exclusion: 7 FP, 9 FN). **xmemory** shows the strongest overall profile with low residual errors in each category (e.g. state: 1 FP, 5 FN; aggregation: 0 FP, 5 FN), indicating isolated gaps rather than systematic failure modes. The creators of each system know their own configurations and optimisation strategies better than any external evaluator. We therefore invite the teams behind Cognee, Mem0, Supermemory, Zep, and any other memory system to run these datasets through their own setups and publish results independently. The datasets are available at https://github.com/xmemory-ai/datasets/tree/main/end_to_end.

8.4 Model real-life use case experiment

The Splitwise experiment evaluates memory in an application-level workflow rather than an isolated extraction or retrieval task. Each write describes a lunch outing in natural language; the underlying event contains structured fields such as date, venue, participants, bill amount, and payer. Each dataset slice contains a sequence of writes followed by reads that ask for direct facts and computed results, including attendees, venues, spending thresholds, and balances between participants.

This benchmark tests extraction from varied language, accumulation of multiple events, aggregation over stored facts, and arithmetic or relational reasoning over retrieved records. It is intentionally structured: the text is generated from a known event model, which makes it especially relevant to schema-based memory systems. It should not be read as a benchmark for open-ended conversational memory in full generality. The cleaned evaluation data and generator are available at <https://github.com/xmemory-ai/datasets/tree/main/splitwise>. Because application-level memory results can vary with prompting, judging, and cached intermediate outputs, we disable LLM-result caching during evaluation and prefer generator-based dataset slices over a single fixed instance.

Table 9: Splitwise dataset measurements, single-answer, LLM as a judge

Product	Accuracy
xmemory	95.2%
Supermemory	73.75%
Cognee	68.0%
Mem0 (graph)	59.1%
Mem0 (no graph)	54.9%
Zep	25.7%

The result suggests that application-level memory quality depends on preserving structured event records, not only on model strength. As additional baselines, we evaluated two file-based Markdown harnesses generated by a frontier model: a transaction-preserving version reached 12% accuracy, while a balance-only version reached 40%. We also compared against customer-facing memory and tool-use harnesses from major frontier-model applications, which reached approximately 92% accuracy on this use case. Those systems are not exposed as reusable API-level memory harnesses, so the comparison should be read as an end-user application comparison, not a developer integration benchmark.

9 Limitations

The results should be interpreted within the scope of this paper. First, the benchmarks are deliberately structured. They target workloads where facts, updates, relations, aggregation, and explicit unknowns matter; they do not claim to measure all forms of open-ended conversational memory. Second, the comparison with Cognée, Mem0, Supermemory, and Zep is a productized-system comparison, not a controlled ablation of every retrieval component. The tested systems differ in implementation details, and their creators may be able to improve results with configurations we did not use. Third, the evaluation relies on LLM judges for some measurements, which introduces judge sensitivity even when protocols are held fixed. Fourth, schema design remains work: it can be imported, authored, or assisted by agents, but the schema is still a contract that must match the intended queries. Finally, validation gates catch many errors but not all semantic errors; incorrect facts can still enter memory if they pass the available validators.

These limitations do not undermine the main claim. They narrow it: for memory workloads that require stable records and stateful computation, explicit schemas and iterative writes provide a more reliable architecture than repeated retrieval-time inference over unstructured text.

10 Conclusions and future work

Schema-grounded memory with an iterative write path achieves 97.10% F1 on the end-to-end benchmark, compared with 80.16%–87.24% across the tested third-party baselines. On structured extraction, iterative decomposition with a judge in the loop reaches 90.42% object-level accuracy, compared with 79.24%–89.24% for the tested frontier structured-output baselines. On the Splitwise application task, `xmemory` reaches 95.2% accuracy. Across these tests, the largest gains appear where memory must preserve state, apply updates, represent relations, compute aggregations, and distinguish absence from missing evidence.

The failure analysis suggests why. Systems that rely primarily on retrieval-time interpretation can retrieve related context but still regress to stale state, assert spurious relations, or miss negative exclusions. Schema-grounded memory does not make errors impossible, but it substantially reduces these failure modes and makes remaining errors more inspectable because updates, relations, and unknowns are represented explicitly. The key architectural move is to shift interpretation from the read path to the write path: validated records are stored once, and reads become constrained queries over those records.

Regarding extraction quality, frontier models exhibit a characteristic accuracy gradient: field-level precision is high, object-level accuracy is lower, and output-level accuracy is lower still. This pattern reveals compositional limits that per-field metrics obscure, and it motivates iterative, judge-in-the-loop extraction. The narrow performance gap observed between GPT-5.4 and GPT-5.5 across these extraction tasks also suggests that model upgrades alone are not the decisive lever; architecture in the write path matters.

Future work includes automating schema inference from intended query patterns, improving conflict handling across multiple input sources, supporting schema evolution with reliable migrations and backfill, adding controlled hybrid-retrieval ablations, and characterising the accuracy-cost tradeoff of iterative extraction across domains.

Acknowledgments

We thank the following people for their contributions and feedback.

System development. Aleksey Shnyukov and Konstantin Leontev.

Advisory and paper review. Alexey Dosovitskiy, Grigory Sapunov and Olga Golovneva.

Early contribution. Vadim Nikulin.

References

- [1] Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, Senjie Jin, Jiejun Tan, Yanbin Yin, Jiongnan Liu, Zeyu Zhang, Zhongxiang Sun, Yutao Zhu, Hao Sun, Boci Peng, Zhenrong Cheng, Xuanbo Fan, Jiabin Guo, Xinlei Yu, Zhenhong Zhou, Zewen Hu, Jiahao Huo, Junhao Wang, Yuwei Niu, Yu Wang, Zhenfei Yin, Xiaobin Hu, Yue Liao, Qiankun Li, Kun Wang, Wangchunshu Zhou, Yixin Liu, Dawei Cheng, Qi Zhang, Tao Gui, Shirui Pan, Yan Zhang, Philip Torr, Zhicheng Dou, Ji-Rong Wen, Xuanjing Huang, Yu-Gang Jiang, and Shuicheng Yan. Memory in the age of ai agents, 2025. URL <https://arxiv.org/abs/2512.13564>. v2, revised 13 Jan 2026.
- [2] Yifan Du, Chongyang Huang, Wayne Xin Zhao, Ji-Rong Wen, et al. Rethinking memory in AI: Taxonomy, operations, topics, and future directions. *arXiv preprint arXiv:2505.00675*, 2025. doi: 10.48550/arXiv.2505.00675. URL <https://arxiv.org/abs/2505.00675>.
- [3] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. URL <https://arxiv.org/abs/2005.11401>.
- [4] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020. URL <https://arxiv.org/abs/2004.04906>.
- [5] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025. doi: 10.48550/arXiv.2504.19413. URL <https://arxiv.org/abs/2504.19413>.
- [6] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019. URL <https://arxiv.org/abs/1908.10084>.
- [7] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 2024. URL <https://arxiv.org/abs/2307.03172>. Published in TACL 2024.
- [8] Darshan Deshpande, Varun Gangal, Hersh Mehta, Anand Kannappan, Rebecca Qian, and Peng Wang. MEMTRACK: Evaluating long-term memory and state tracking in

- multi-platform dynamic agent environments. *arXiv preprint arXiv:2510.01353*, 2025. doi: 10.48550/arXiv.2510.01353. URL <https://arxiv.org/abs/2510.01353>.
- [9] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. LoCoMo: Evaluating very long-term conversational memory of LLM agents. *arXiv preprint arXiv:2402.17753*, 2024. doi: 10.48550/arXiv.2402.17753. URL <https://arxiv.org/abs/2402.17753>.
 - [10] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley, 2 edition, 2006. URL <https://onlinelibrary.wiley.com/doi/book/10.1002/047174882X>.
 - [11] Claude E. Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27(3):379–423, 1948. URL <https://people.math.harvard.edu/~ctm/home/text/others/shannon/entropy/entropy.pdf>.
 - [12] Shizhe He, Avanika Narayan, Ishan S. Khare, Scott W. Linderman, Christopher Ré, and Dan Biderman. An information theoretic perspective on agentic system design. *arXiv preprint arXiv:2512.21720*, December 2025. doi: 10.48550/arXiv.2512.21720. URL <https://arxiv.org/abs/2512.21720>.
 - [13] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020. URL <https://arxiv.org/abs/1910.13461>.
 - [14] Jingqing Zhang, Yao Zhao, Mohammad Saleh, and Peter J. Liu. PEGASUS: Pre-training with extracted gap-sentences for abstractive summarization. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020. URL <https://arxiv.org/abs/1912.08777>.
 - [15] Adam Tauman Kalai, Ofir Nachum, Santosh S. Vempala, and Edwin Zhang. Why language models hallucinate. *arXiv preprint arXiv:2509.04664*, 2025. doi: 10.48550/arXiv.2509.04664. URL <https://arxiv.org/abs/2509.04664>.
 - [16] Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with BERT. *arXiv preprint arXiv:1901.04085*, 2019. URL <https://arxiv.org/abs/1901.04085>.
 - [17] Muhammad Ahmed Mohsin, Muhammad Umer, Ahsan Bilal, Zeeshan Memon, Muhammad Ibtisam Qadir, Sagnik Bhattacharya, Hassan Rizwan, Abhiram R. Gorle, Maahe Zehra Kazmi, Ayesha Mohsin, Muhammad Usman Rafique, Zihao He, Pulkit Mehta, Muhammad Ali Jamshed, and John M. Cioffi. On the fundamental limits of LLMs at scale. *arXiv preprint arXiv:2511.12869*, 2025. URL <https://arxiv.org/abs/2511.12869>.
 - [18] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024. URL <https://arxiv.org/abs/2404.16130>.
 - [19] Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, Victor Zhong, Caiming Xiong, Ruoxi

- Sun, Qian Liu, Sida Wang, and Tao Yu. Spider 2.0: Evaluating language models on real-world enterprise text-to-SQL workflows, 2024. URL <https://arxiv.org/abs/2411.07763>. ICLR 2025 Oral.
- [20] Derong Xu, Wei Chen, Wenjun Peng, Chao Zhang, Tong Xu, Xiangyu Zhao, Xian Wu, Yefeng Zheng, Yang Wang, and Enhong Chen. Large language models for generative information extraction: A survey. *arXiv preprint arXiv:2312.17617*, 2024. URL <https://arxiv.org/abs/2312.17617>.
 - [21] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegraffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. *arXiv preprint arXiv:2303.17651*, 2023. URL <https://arxiv.org/abs/2303.17651>.
 - [22] Elliot Meyerson, Giuseppe Paolo, Roberto Dailey, Hormoz Shahrzad, Olivier Francon, Conor F. Hayes, Xin Qiu, Babak Hodjat, and Risto Miikkulainen. Solving a million-step LLM task with zero errors. *arXiv preprint arXiv:2511.09030*, 2025. doi: 10.48550/arXiv.2511.09030. URL <https://arxiv.org/abs/2511.09030>.
 - [23] John Dagdelen, Alexander Dunn, Sanghoon Lee, Nicholas Walker, Andrew S. Rosen, Gerbrand Ceder, Kristin A. Persson, and Anubhav Jain. Structured information extraction from scientific text with large language models. *Nature Communications*, 15(1):1418, 2024. URL <https://www.nature.com/articles/s41467-024-45563-x>.
 - [24] Haolun Wu, Ye Yuan, Liana Mikaelyan, Alexander Meulemans, Xue Liu, James Hensman, and Bhaskar Mitra. Learning to extract structured entities using language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 6817–6834. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.emnlp-main.388. URL <https://aclanthology.org/2024.emnlp-main.388/>.
 - [25] Yujie Luo, Xiangyuan Ru, Kangwei Liu, Lin Yuan, Mengshu Sun, Ningyu Zhang, Lei Liang, Zhiqiang Zhang, Jun Zhou, Lanning Wei, Da Zheng, Haofen Wang, and Huajun Chen. OneKE: A dockerized schema-guided LLM agent-based knowledge extraction system. In *Companion Proceedings of the ACM Web Conference 2025 (WWW Companion '25)*, 2025. doi: 10.1145/3701716.3715189. URL <https://arxiv.org/abs/2412.20005>.
 - [26] Kanghee Park, Jiayu Wang, Taylor Berg-Kirkpatrick, Nadia Polikarpova, and Loris D’Antoni. Grammar-aligned decoding. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL <https://arxiv.org/abs/2405.21047>.
 - [27] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. Why and where: A characterization of data provenance. In *International Conference on Database Theory (ICDT)*, 2001. URL <https://homepages.inf.ed.ac.uk/opb/papers/ICDT2001.pdf>.
 - [28] Brandon Hong et al. Context rot: How increasing input tokens impacts LLM performance. Technical report, Chroma, 2025. URL <https://research.trychroma.com/context-rot>.
 - [29] YAML Language Development Team. YAML ain’t markup language (YAML) version 1.2.2. <https://yaml.org/spec/1.2.2/>, 2021.
 - [30] JSON Schema Authors. JSON schema: A media type for describing JSON documents (draft 2020-12). <https://json-schema.org/draft/2020-12/json-schema-core>, 2020.

- [31] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022. URL <https://arxiv.org/abs/2210.03629>.
- [32] Uta Störl, Meike Klettke, and Stefanie Scherzinger. NoSQL schema evolution and data migration. In *Proceedings of the 23rd International Conference on Extending Database Technology (EDBT)*, 2020. URL https://openproceedings.org/2020/conf/edbt/paper_T4.pdf.
- [33] Alberto Hernández Chillón, Meike Klettke, Diego Sevilla Ruiz, and Jesús García Molina. A generic schema evolution approach for NoSQL and relational databases. *IEEE Transactions on Knowledge and Data Engineering*, 2024. URL https://epub.uni-regensburg.de/77266/1/A_Generic_Schema_Evolution_Approach_for_NoSQL_and_Relational_Databases.pdf.
- [34] Jonas Mueller Hui Wen Goh. LLM structured output benchmarks are riddled with mistakes, 2025. URL <https://cleanlab.ai/blog/structured-output-benchmark/>. Accessed: 2026-04-16.
- [35] Topoteretes. Cognee github repository and readme. <https://github.com/topoteretes/cognee>, 2026. Accessed 2026-04-22.
- [36] Mem0. Mem0 documentation: Build with mem0. <https://docs.mem0.ai/introduction>, 2026. Accessed 2026-04-22.
- [37] Supermemory. Supermemory documentation: Overview — what is supermemory? <https://supermemory.ai/docs/intro>, 2026. Accessed 2026-04-22.
- [38] Zep. Zep documentation and platform overview. <https://www.getzep.com>, 2026. Accessed 2026-04-24.
- [39] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Long-MemEval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024. doi: 10.48550/arXiv.2410.10813. URL <https://arxiv.org/abs/2410.10813>.
- [40] Letta. Benchmarking ai agent memory. <https://www.letta.com/blog/benchmarking-ai-agent-memory>, 2026. Accessed 2026-04-24.
- [41] snap-research and community contributors. Locomo issue discussion: Dataset label quality estimate. <https://github.com/snap-research/locomo/issues/27#issuecomment-3921992262>, 2025. Accessed 2026-04-24.

A Appendix

A.1 Information-theoretic intuition

Extracting structured facts from language is a transition from a high-entropy representation to a low-entropy one. Natural language admits many paraphrases for the same underlying meaning, and it tolerates ambiguity, omission, and implicit context. A schema does not. It fixes field names, types, and constraints, and it excludes ambiguity by construction.

Extraction therefore requires irreversible choices: which entities exist, which fields apply, which values are correct, and which information is irrelevant. Each choice collapses alternatives. If the system misinterprets context or misses a qualifier, the discarded alternative cannot be recovered downstream because the stored representation has already committed.

This is a different kind of risk than the compression loss in Section 1.2. Compression discards information. Extraction also commits to one interpretation of what remains. That makes write-path errors more dangerous than read-path errors: a read error affects one answer, while a write error corrupts stored memory and propagates into future reasoning and retrieval.

One way to frame the “entropy jump” is to treat extraction as inferring a schema-shaped record Y from an input X . The residual ambiguity at commit time is captured by the conditional entropy $H(Y | X)$. Iterative extraction introduces intermediate constraints and signals C (detected fields, validators, cross-checks), and conditioning can only reduce uncertainty:

$$H(Y | X, C) \leq H(Y | X).$$

The reduction $\Delta H = H(Y | X) - H(Y | X, C) = I(Y; C | X)$ captures how much schema constraints and validation shrink the space of plausible interpretations [10–12].

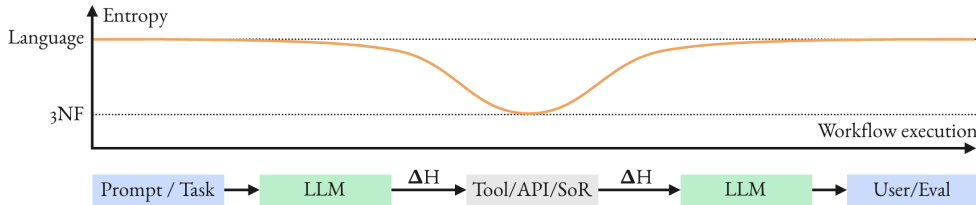


Figure 7: Entropy jump: each time a pipeline uses a tool, calls an API or queries System of Record it makes an entropy jump with potential information losses.

Any system that extracts structured memory from language must confront the difficulty of mapping a high-entropy input space into a low-entropy representation without losing critical information.

A.2 Overview of existing memory benchmarks: LoCoMo and LongMemEval

This appendix summarises our perspective regarding the existing benchmarks—LoCoMo and LongMemEval [9, 39]. We are not the first team to challenge widely adopted approaches to evaluate quality of memory systems. For example, the Letta team publicly questioned aspects of LoCoMo benchmarking setup and interpretation [40].

In our view, the core issue is that LoCoMo and LongMemEval [9, 39] frequently mix memory-function evaluation with non-trivial reasoning. Many questions are not pure memory checks (“is the fact stored and retrievable?”) but compound tasks that include reasoning, planning, or synthesis on top of retrieval.

Relatedly, common question labels such as *multi-hop*, *temporal*, and *open domain* can be ambiguous in practice. A single question may plausibly fit multiple labels at once, and borderline cases are frequent. For example, LoCoMo has these three questions within one conversation conv-30:

```
{
  "question": "What do Jon and Gina both have in common?",
  "answer": "They lost their jobs and decided to start their own businesses."
},
...
{
  "question": "What is Gina's favorite style of dance?",
  "answer": "Contemporary"
},
...
{
  "question": "What is Jon's favorite style of dance?",
  "answer": "Contemporary"
},
```

These three questions and their assigned labels illustrate the issue: the label set is not comprehensive enough to capture all relevant dimensions of the task. They also show practical ambiguity, since equally plausible interpretations can map the same question to different label categories.

A deeper problem is that some questions in these benchmarks have no factual answer grounded in the conversation at all. The evidence utterance for the financial status question below contains no statement about wealth or income; the expected answer is an inference the model must draw entirely from indirect cues. Similarly, the Dr. Seuss question asks whether a character *would likely* own certain books—a probabilistic prediction about a fictional person’s behaviour, not a fact that was ever stored. These are reasoning tasks presented as memory tasks. Conflating the two makes it impossible to separate a system’s ability to retain and retrieve facts from its ability to perform commonsense or probabilistic inference, and it systematically favours systems that invest in reasoning rather than in reliable memory representation.

```
{
  "question": "What might John's financial status be?",
  "answer": "Middle-class or wealthy",
  "evidence": [
    "D5:5"
  ],
  "category": 3
},
...
{
  "speaker": "John",
  "dia_id": "D5:5",
  "text": "It's definitely isn't, Maria. My kids have so much and others don't.
    We really need to do something about it."
},
...
{
  "question": "Would Caroline likely have Dr. Seuss books on her bookshelf?",
  "answer": "Yes, since she collects classic children's books",
  "evidence": [
    "D6:9"
```



```

    ],
    "category": 3
  },
  ...
  {
    "speaker": "Caroline",
    "dia_id": "D6:9",
    "text": "I've got lots of kids' books- classics,
    stories from different cultures, educational books,
    all of that. What's a favorite book you remember from your childhood?"
  },

```

LoCoMo also has a noticeable number of erroneous labels: a reported estimate puts label errors at about 6.4% [41]. Many of these issues we identified independently before reviewing that report. LongMemEval github repository also has a few reports of label errors, similar in nature to LoCoMo ones. Taken together, these factors make reproducibility and fair comparison of memory systems difficult, and they blur the quality of the memory harness with the quality of the models used in extraction, retrieval, and LLM-judge components.

Given this, we propose a targeted benchmark approach focused on memory functions themselves: minimal or no reasoning required in read tasks, unambiguous labels, and straightforward analysis of where quality is lost (extraction, write, or read). This is the design principle behind the end-to-end evaluation protocol described in Section 8.3.